Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Создание модуля бизнес-логики

otus.ru

Правила вебинара

Активно
участвуем

Off-topic обсуждаем
в TG OTUS Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП	DSL	
Модуль 2: Расширенные возможности Kotlin	Корутины	Sequence, Flow	Multiplatform	Interop	Gradle	
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	DevOps	Архитектура		
Модуль 4: Разработка API	OpenAPI	CollabAPI	Модели и мапперы	Testing	Monitoring	
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репози- торий	Конкурен- ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде		
Модуль 9: Проектная работа	Обзор экосист.	Консуль- тация	Защита			

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План полета

- Шаблон CoR
- Контекст, модели, статус
- Интеграция в проект
- Рефлексия

Цели вебинара

После занятия вы сможете

1. Реализовать бизнес-логику используя шаблон “Цепочка ответственности”
2. Строить гибкую логику и описывать бизнес-процессы в приложении
3. Интегрировать бизнес-логику в приложение

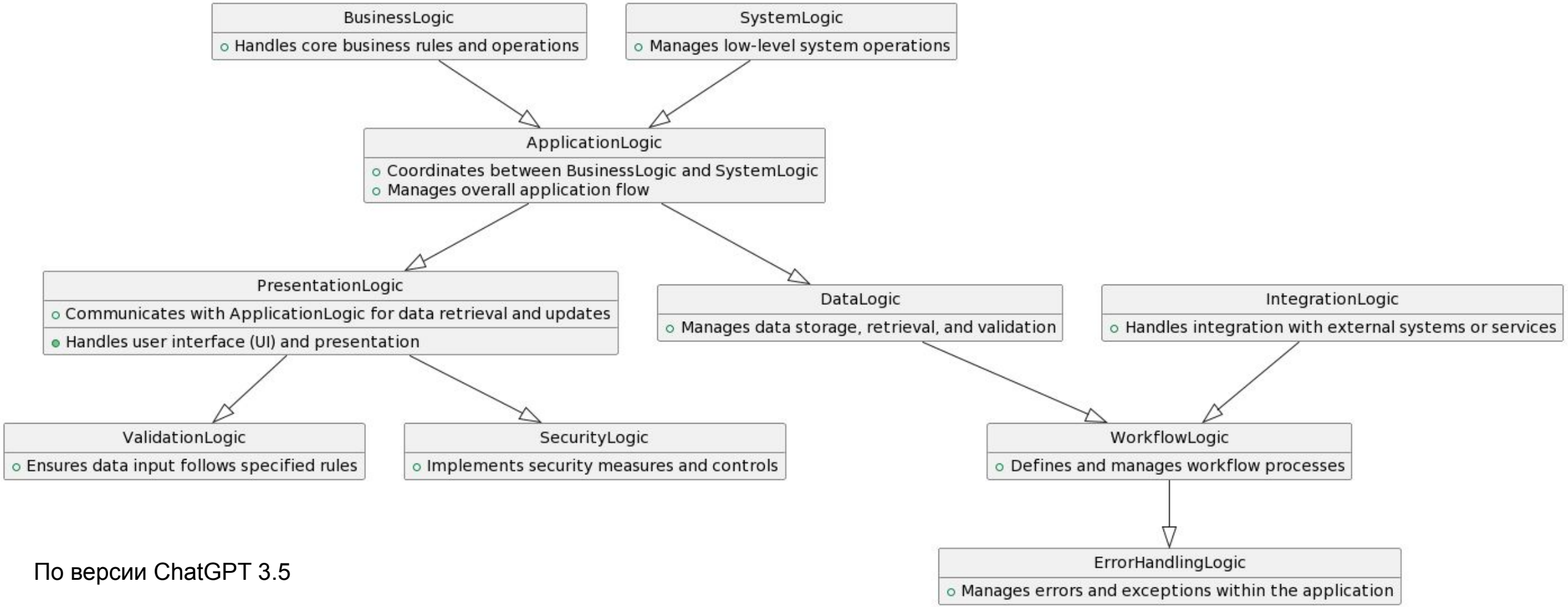
Смысл

Зачем вам это уметь

1. Бизнес-процессы часто меняются, только гибкая логика позволит обеспечить техническое сопровождение проекта за приемлемые ресурсы
2. CoR гораздо гибче чем маркетинговые проекты типа BPMN и BPEL и проще Модели предметной области
3. Создавать качественный, легко управляемый и легко читаемый код

Еще раз о бизнес-логике

Логика информационной системы



По версии ChatGPT 3.5

Особенности Бизнес-логики

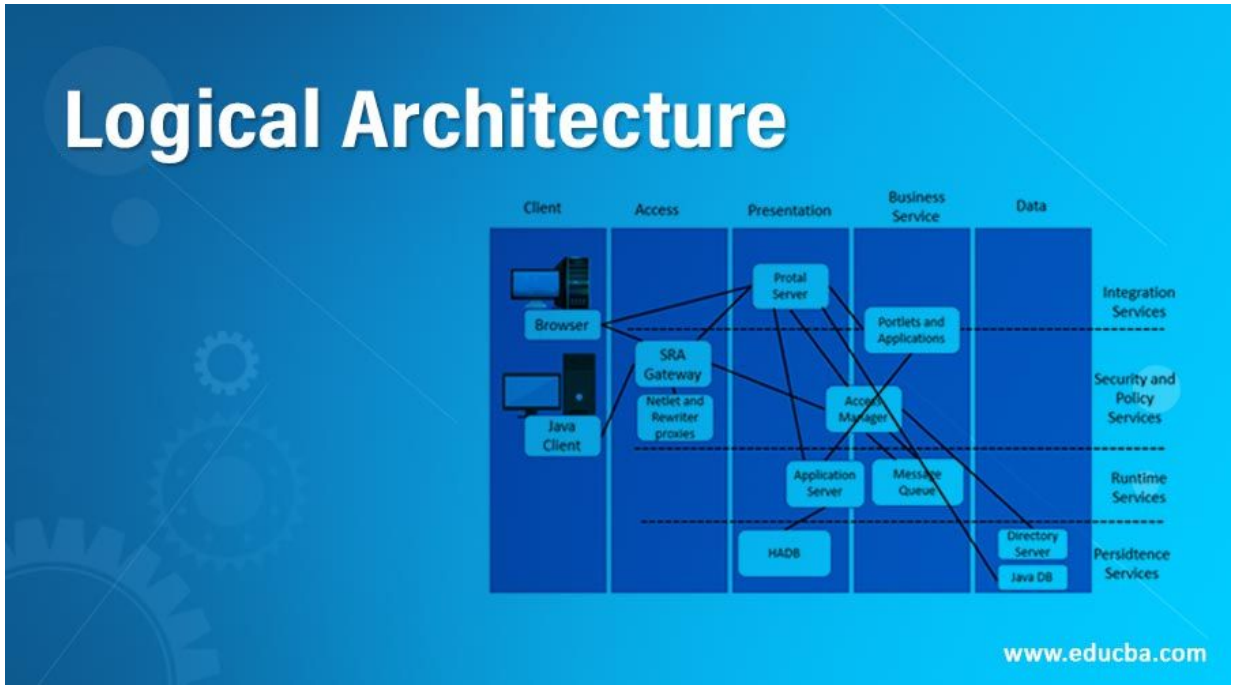
1. **Реализация Бизнес-Правил:**
   Бизнес-логика отражает бизнес-правила, которые определяют, как система должна вести себя в рамках конкретного бизнес-процесса или домена.
2. **Независимость от Технических Деталей:**
   Бизнес-логика стремится быть независимой от технических деталей реализации, что обеспечивает ее переносимость и гибкость.
3. **Управление Бизнес-Процессами:**
   Включает в себя логику, связанную с управлением бизнес-процессами, такими как обработка заказов, управление инвентарем или обработка транзакций.
4. **Доступ к Данным:**
   Бизнес-логика определяет, как система взаимодействует с данными, какие операции выполняются, и какие данные требуются для принятия решений.
5. **Обеспечение Консистентности:**
   Задача бизнес-логики включает в себя обеспечение консистентности данных и согласованности действий системы в контексте бизнес-правил.
6. **Поведенческая Часть Системы:**
   Чаще всего, бизнес-логика определяет поведение системы и то, как она взаимодействует с внешним миром в соответствии с бизнес-требованиями.
7. **Тестируемость и Поддерживаемость:**
   Хорошо структурированная бизнес-логика обеспечивает легкость тестирования и обслуживания, что является важным фактором для долгосрочной эффективной работы системы.
8. **Система Знаний Организации:**
   Часто бизнес-логика воплощает в себе знание о том, как бизнес процесс организации должен быть реализован в программной форме.

Любые бизнес-требования могут меняться, потому логика должна быть гибкой

Подходы к построению бизнес-логики

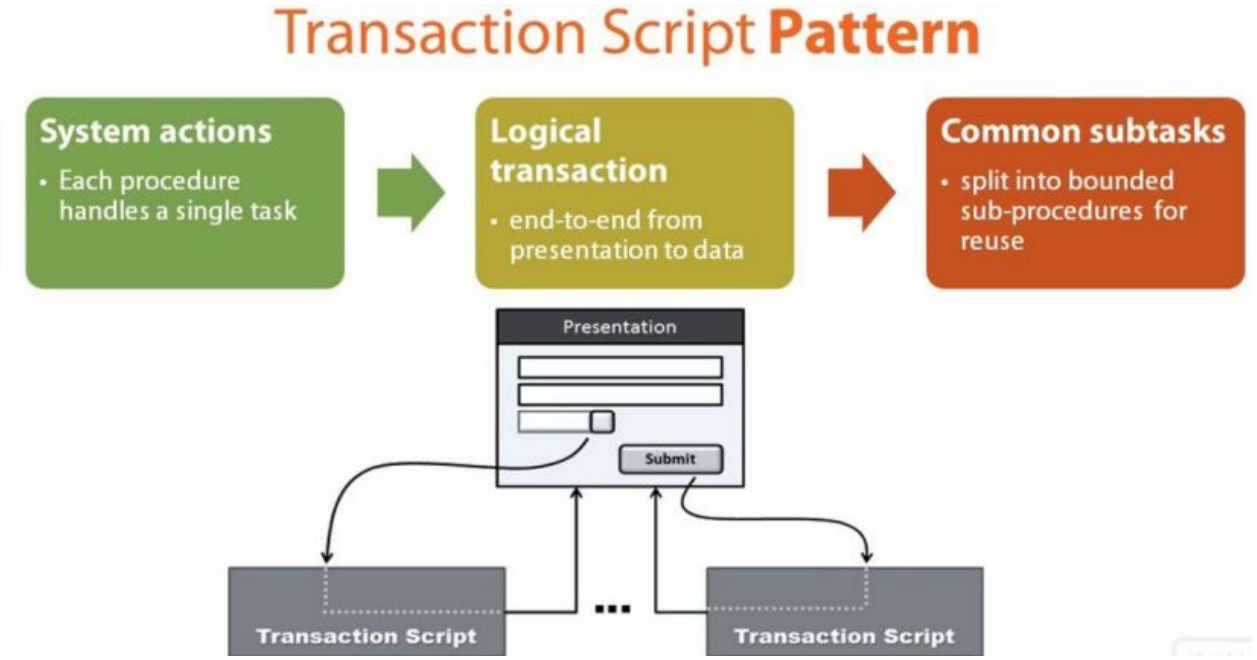
Подходы к описанию бизнес-логики

- Транзакционный подход. Реализуем каждую функцию, смотрим что получилось, исправляем ошибки, переделываем для расширения
- Проектирование БД. Представляем что нужно хранить в БД, проектируем ее, обвешиваем обработками
- Проектирование предметной области. Формируем понятия, связи, выстраиваем взаимодействия, реализуем
- Гибкая логика. Исходим из того, что любая функциональность может измениться или расшириться



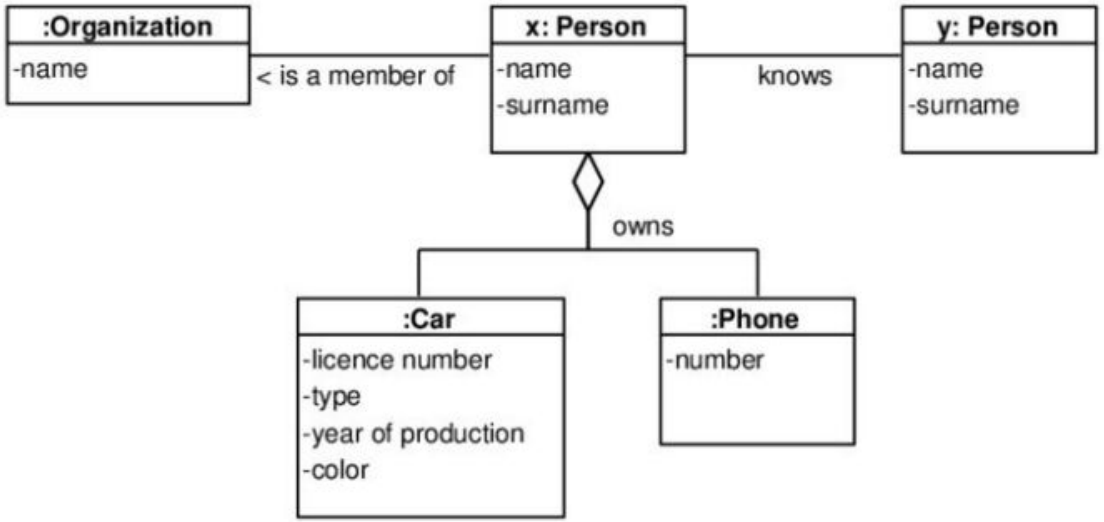
Транзакции

- Собственный сценарий для каждой бизнес-транзакции
- Непосредственный доступ к данным
- Прост в понимании
- Низкая гибкость
- Дублирование кода
- Проблемы решаются декомпозицией сценариев



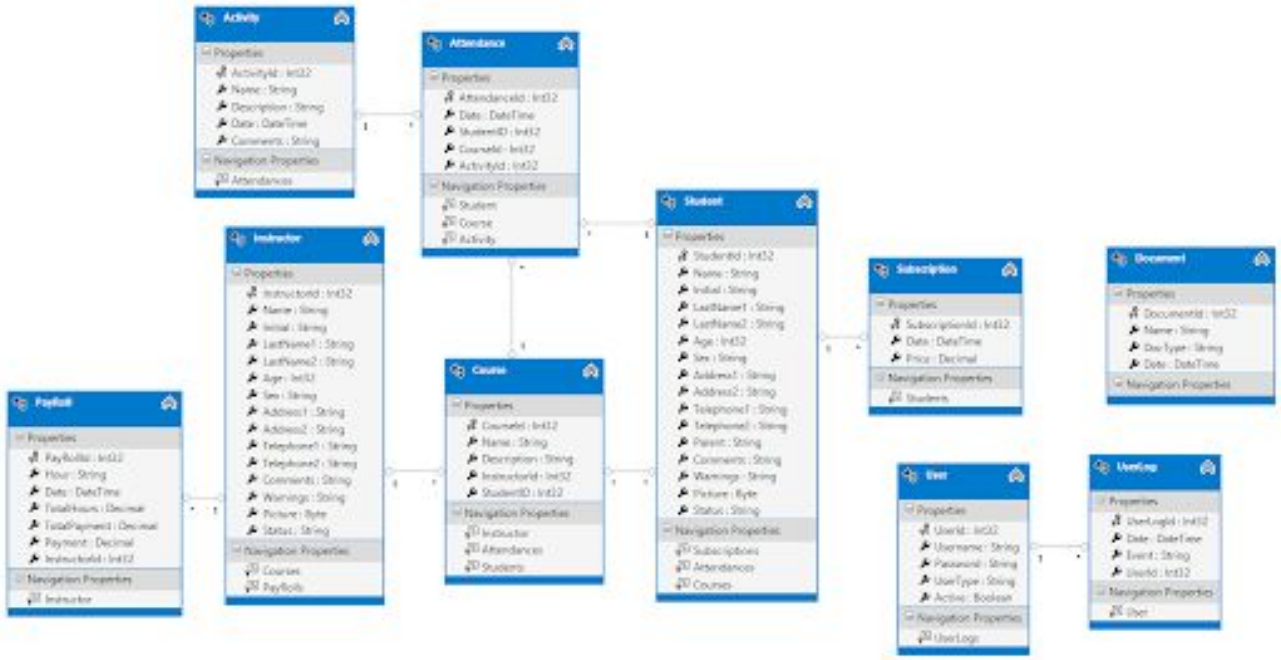
Модель предметной области

- Объекты представляют бизнес-сущности
- Доступ к данным посредством объектов
- Низкая гибкость
- Исключает дублирования кода
- Субъективность



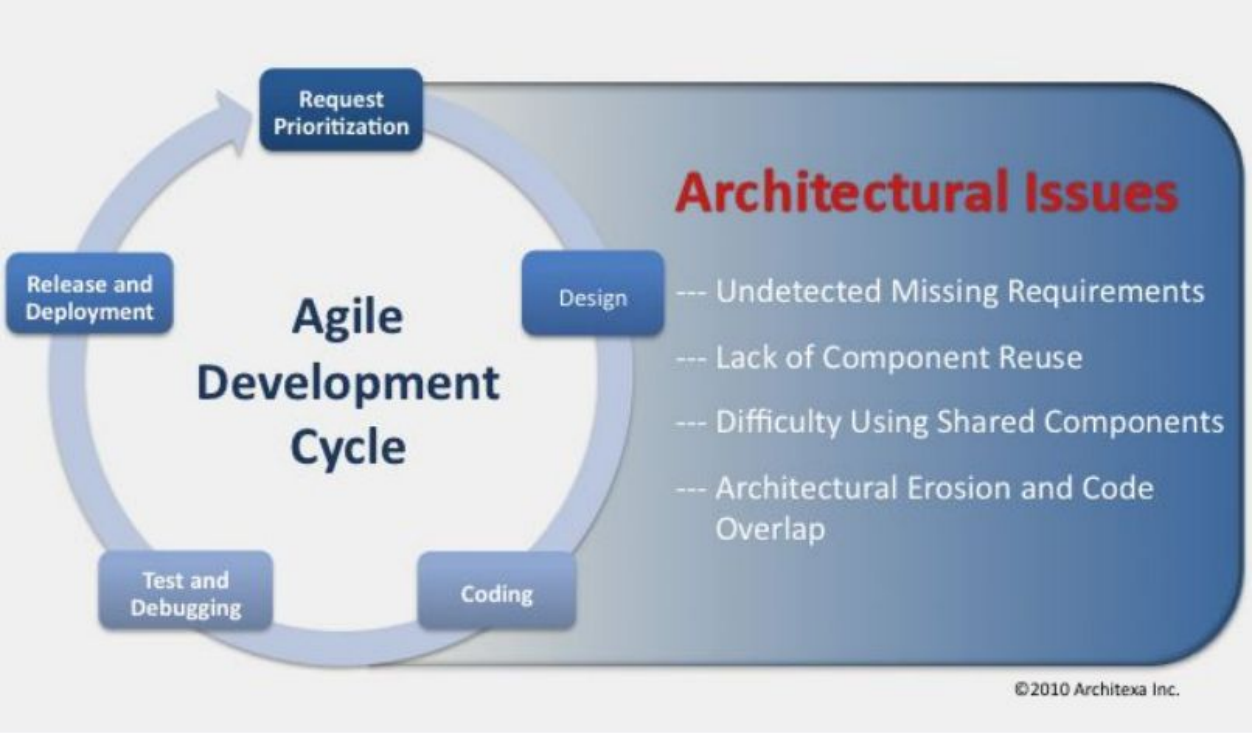
DB First

- Навязываются некоторыми фреймворками (например, Yii)
- Правильно построенная модель хранения помогает быстро сгенерировать все приложение
- Отрыв от бизнес-требований
- Отсутствие гибкости



Гибкая логика

- Активно внедряется во все сферы
- Требует закладывать возможности расширения и изменения на этапе проектирования
- Требует некоторый оверхед на этапе проектирования
- Более дешева при реализации и поддержке

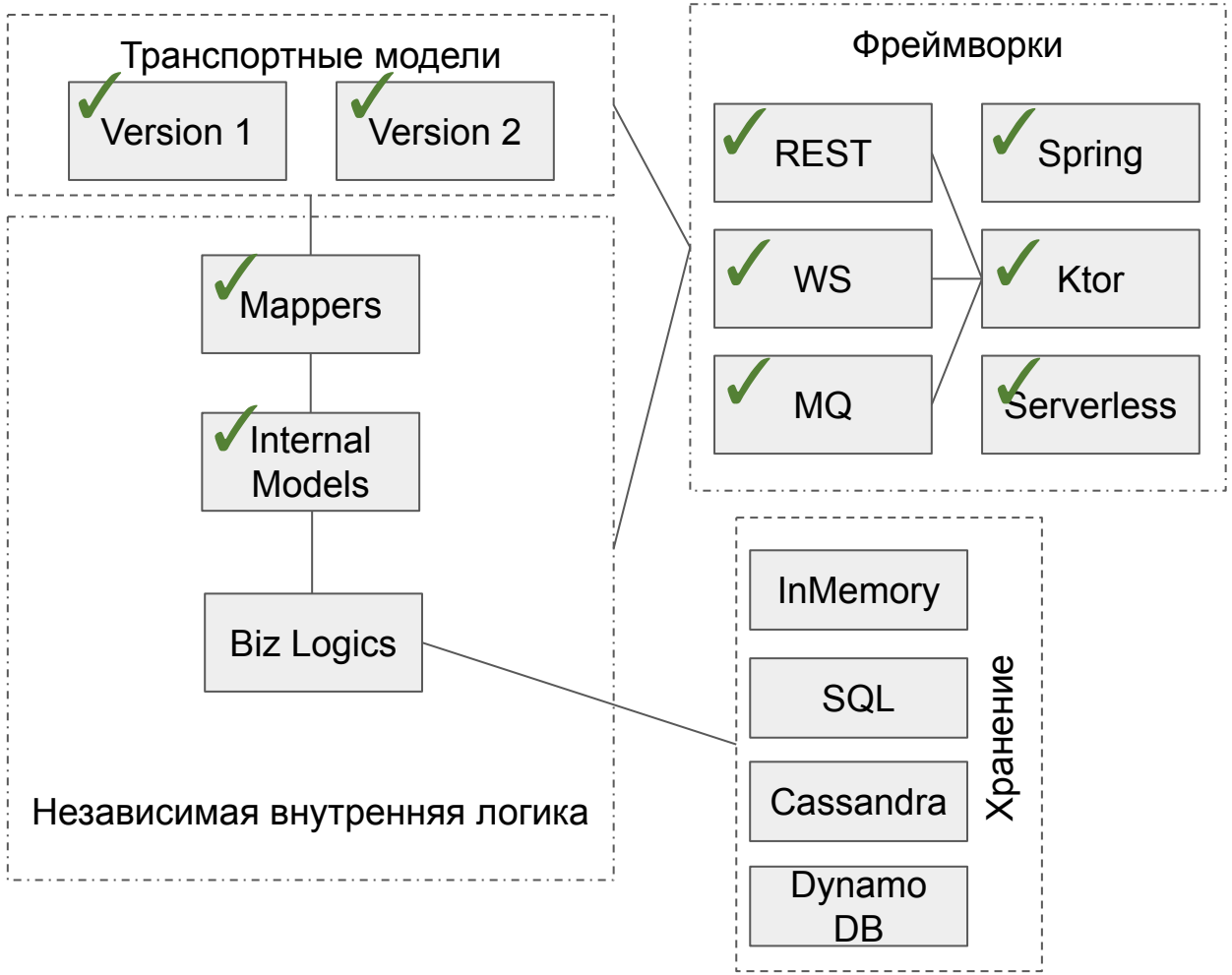


Особенности бизнес-логики

Принципы целевой архитектуры

Особенности:

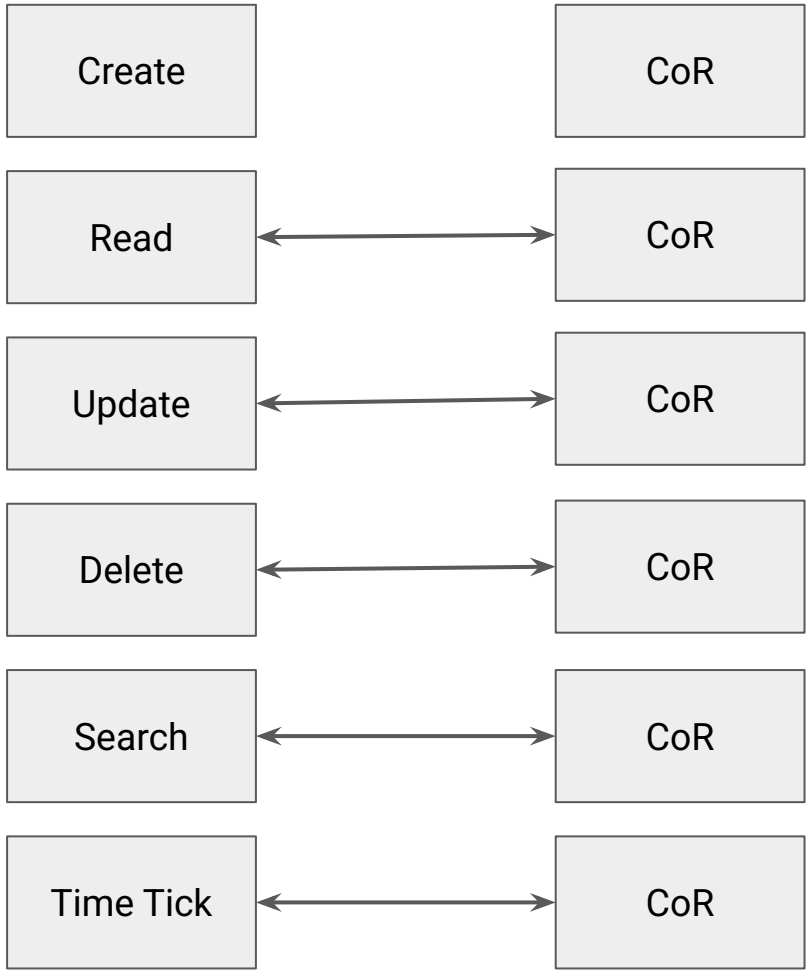
1. Бизнес-логика - внутренняя логика
2. Не показываем наружу, но …
3. Контракты определяются бизнес-(функциональными) требованиями
4. Бизнес-логика не зависит от конкретных реализаций, конкретных БД, API и прочих инструментов



Event driven

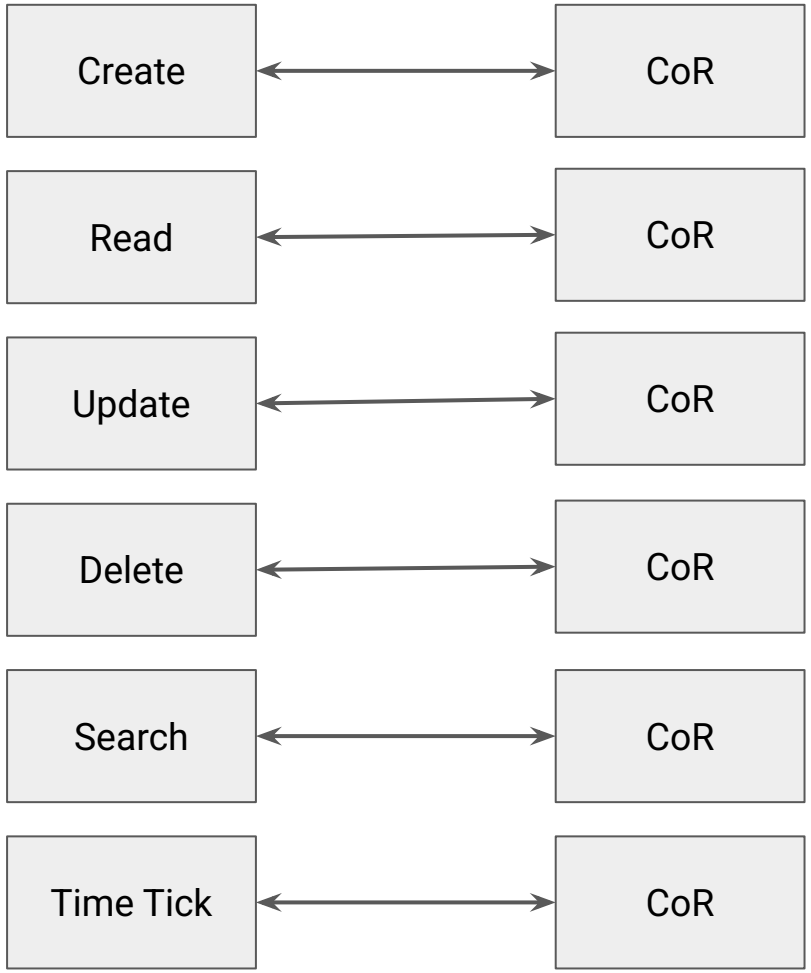
Бизнес-логика отражает событийный подход

1. В контроллере ловим события
2. Каждое событие обрабатываем с помощью CoR
3. Возможные события:
   1. Запрос REST
   2. Сообщение из брокера сообщений
   3. Таймер
   4. И др.



Требования к бизнес-цепочке

1. Это управляющий шаблон, а не реализующий. Все реализации конкретных функций должны быть вынесены в функциональные классы
2. Типичный состав цепочки:
   a. Инициализация
   b. Обработка стабов
   c. Валидация
   d. Работа с БД и др. репозиториями
   e. Работа с машиной состояний
   f. Работа с ML
   g. Подготовка ответа
   h. Завершение
3. Для каждого источника событий своя цепочка
4. Цепочки могут объединяться

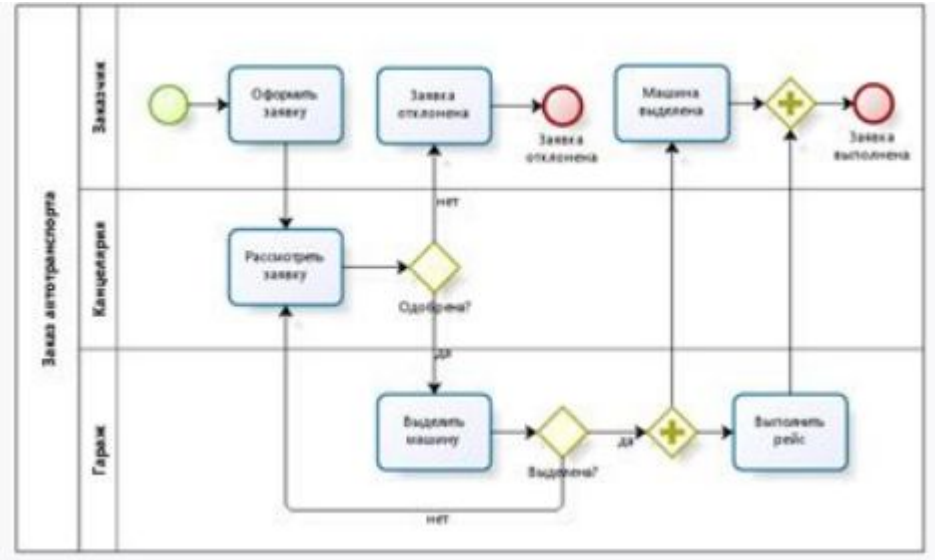


Как описать гибкую логику

Business Process Model and Notation

Business Process Model and Notation - BPMN, BPMS, BPM:

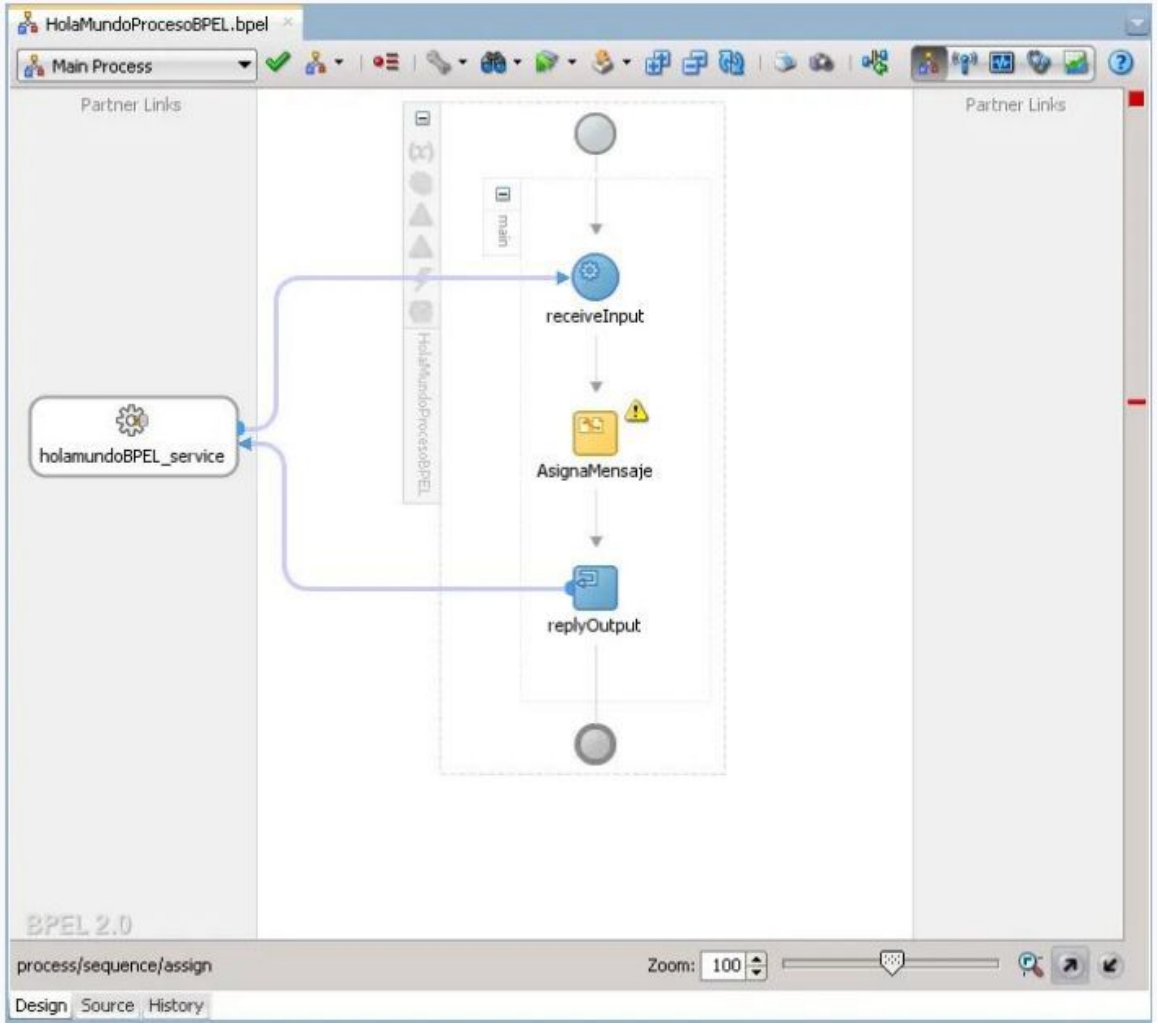
1. Представители:
   a. Camunda,
   b. jBOSS BPM,
   c. Adobe LifeCycle,
   d. etc
2. Концепция: аналитик рисует визуально бизнес-процесс, а реализация силами разработчиков
3. Преимущества: формализует и упорядочивает разработку
4. Недостатки
   a. У аналитика нет квалификации разработчика
   b. Разработчику неудобно работать
   c. Клиенты плохо понимают нотации



BPEL

Business Process Execution Language - BPEL:

1. Представители:
   a. jBPM,
   b. Apache ODE,
   c. BizTalk Server,
   d. etc
2. Концепция: разработчик сам строит бизнес процесс на собственном DSL
3. Недостатки
   a. XML



Элементы бизнес-логики

Контекст

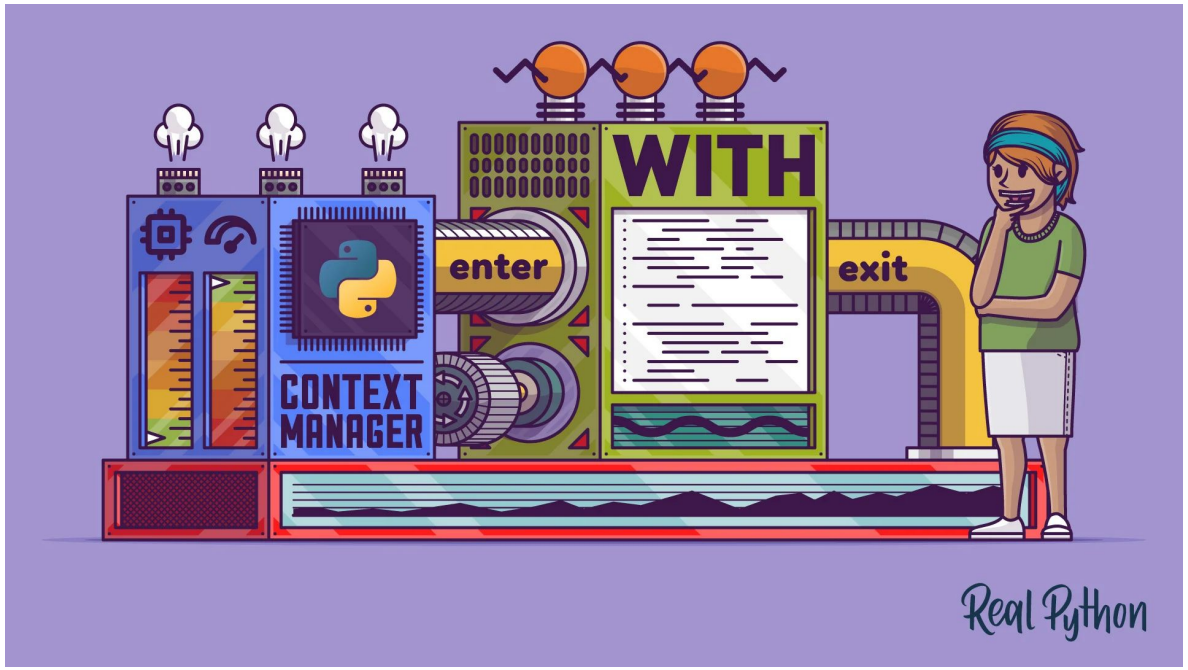
Содержит всю необходимую информацию для обработки запроса

- Настройки системы
- Входящие данные
  - requestId
  - данные запроса
  - метрические данные
  - отладочные команды
- Иходящие данные
  - данные для ответа
  - ошибки
- Статус
- Промежуточные данные

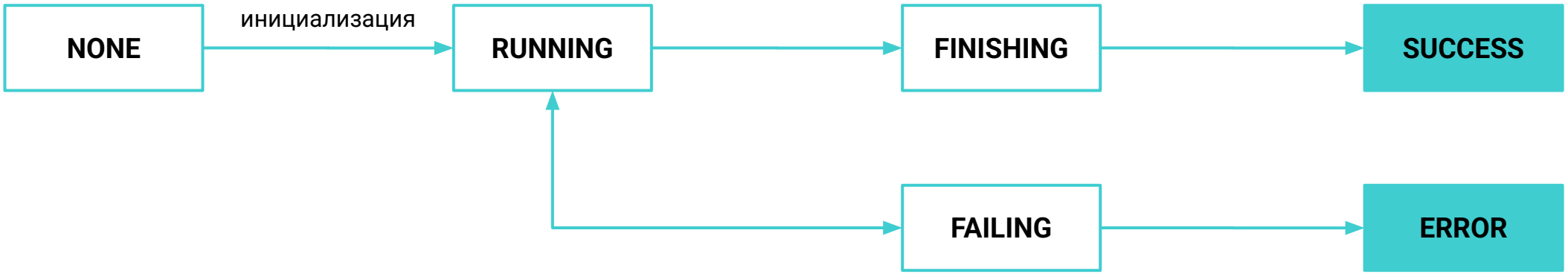
```
data class MpContext(
   var onRequest: String = "",
   var requestAdId: AdIdModel = AdIdModel.NONE,
   var requestAd: AdModel = AdModel(),
   var responseAd: AdModel = AdModel(),
   var requestPage: PaginatedModel = PaginatedModel(),
   var responsePage: PaginatedModel = PaginatedModel(),
   var responseAds: MutableList<AdModel> =
mutableListOf(),
   var errors: MutableList<MpError> = mutableListOf(),
   var status: CorStatus = CorStatus.STARTED,
)
```

Требования к контексту

1. Является исключительно внутренним классом и не может быть опубликован
2. Представляет данные в удобном для внутренней логики виде
3. Избегаем интерфейсов и наследования во внутренних моделях, используем расширяемость (добавляем необходимые поля)
4. Могут использоваться “под-контексты”
5. Крайности - на каждый чейн свой контекст и один контекст на всю систему. Необходимо избегать крайностей



Статусы и статусная модель




```
enum class CorStatus {
   NONE,
   RUNNING,
   FINISHING,
   FAILING,
   ~~SUCCESS~~,
   ~~ERROR~~;

   val isError: Boolean
       get() = this in setOf(FAILING, ~~ERROR~~)
}
```

```
data class MpContext(
   var status: CorStatus = CorStatus.NONE,
   ...
)
```

Проект может иметь свои статусы

Статусы позволяют включать или отключать некоторые обработки.

Статусная модель дает нам возможность ветвления логики

Обработка ошибок

```
data class MpError(
   val code: String = “”,
   val group: Group = “”,
   val field: String = “”,
   val level: Level = ERROR,
   val message: String = “”,
) {
   enum class Group(val alias: String) {
       NONE(""),
       SERVER("internal-server")
   }

   enum class Level(val weight: Int) {
       FATAL(90),
       ERROR(70),
       WARNING(40),
       INFO(20);

       val isError: Boolean
           get() = weight >= ERROR.weight

       val isWarning: Boolean
           get() = this == WARNING
   }
}
```

```
data class MpContext(
   var errors: MutableList<MpError> = mutableListOf(),
   ...
)

// Usage example
errors.add(...)
status = CorStatus.FAILING
```

Управляем обработкой ошибок:

- (**errors.add(...)**):
  - клиенту описание ошибки и что он может сделать для ее устранения
  - логика обработки ошибки (редко)
- (**logger.error(...)**)
  - разработчику предоставить всю информацию об ошибке

Stub cases

```
enum class MpStubCase {
    NONE,

    AD_CREATE_SUCCESS,
    AD_READ_SUCCESS,
    AD_UPDATE_SUCCESS,
    AD_DELETE_SUCCESS,
    OFFERS_SUCCESS,
}
```

```
data class MpContext(
    var stubCase: MpStubCase = MpStubCase.NONE,
    ...
)
```

Стабы позволяют тестировать клиентский код без реальных запросов в БД или к сторонним сервисам

Интеграционное тестирование

Список стабов специфицируем в API.

Интеграция бизнес-логики

Интеграция бизнес-логики

1. Избегаем любых зависимостей, кроме common и базовых библиотек типа coroutines, kotlinx-datetime
2. Все конкретные реализации через адаптеры
3. В бизнес-логике решаем ЧТО ДЕЛАТЬ, а не ~~КАК ДЕЛАТЬ~~
4. Подключаем либо в фреймворке, либо в модуле сервиса

LIVE

Рефлексия

Проверка усвоенного материала

1.	Какие основные статусы требуется предусмотреть?
2.	Что должен содержать класс ошибок?
3.	Что должна содержать и что не должна содержать типичная бизнес-цепочка?

2 мин

Основные тезисы

1.	Бизнес-логика: правила, поведение, бизнес-процессы, консистентность и пр.
2.	Отражает Event Driven подход. Для каждого источника событий собственная цепочка
3.	Элементы бизнес-логики: CoR, контекст, статусная модель, обработка ошибок, стабы
4.	Типичная бизнес-цепочка бизнес-логики: инициализация, валидаторы, заглушки, работа с БД, подготовка ответов
5.	CoR - управляющий шаблон, реализации в отдельных модулях

Домашнее задание

1. Сделать модуль бизнес-логики для своего выпускного проекта
2. Реализовать заглушки для запросов с помощью шаблона CoR
3. Выполнить валидацию в модуле бизнес-логики
4. Сделать автотесты для бизнес-логики

Рефлексия

С какими основными мыслями и инсайтами уходите с вебинара?

Какой самый “глупый” вопрос у вас остался?

Цели вебинара

Проверка достижения целей

1. Реализовать бизнес-логику используя шаблон “Цепочка ответственности”
2. Строить гибкую логику и описывать бизнес-процессы в приложении
3. Интегрировать бизнес-логику в приложение

Список материалов для изучения

- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/202505-ok-marketplace
- https://refactoring.guru/ru/design-patterns/chain-of-responsibility
- https://ru.wikipedia.org/wiki/Цепочка_обязанностей
- BPMN: https://ru.wikipedia.org/wiki/BPMN
- BPEL: https://ru.wikipedia.org/wiki/BPEL

Следующий вебинар

Модуль 6: Бизнес-логика проекта

Шаг валидации в бизнес-цепочке

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары